

□ LINKED LISTS

The Complete Deep Dive

With C / C++ and Python Code Examples

“And He is with you wherever you are.”

— Surah Al-Hadid, 57:4

□ SECTION 1 — What Is a Linked List?

The Treasure Hunt Analogy

Imagine a treasure hunt. You start at clue #1. Clue #1 does NOT tell you where all the clues are — it only tells you where clue #2 is. Clue #2 tells you where clue #3 is. The last clue says “you’re done” (which in code is NULL or None).

That is exactly what a Linked List is.

Definition

A Linked List is a chain of NODES. Each node holds two things: 1. DATA — the actual value (a number, a string, anything) 2. A POINTER — the address in memory of the next node

Unlike arrays, the nodes are NOT sitting next to each other in memory. They can be scattered anywhere. They find each other purely through pointers.

Arrays vs Linked Lists — Why Does It Matter?

When you create an array, your computer reserves a contiguous (side-by-side) block of memory. This is great for speed, but causes problems when you need to grow it or insert in the middle.

```
Array in memory (contiguous):
Address: [100][101][102][103][104]
Values:  [ A ][ B ][ C ][ D ][ E ]

Insert X at position 2 requires shifting:
Before: [ A ][ B ][ C ][ D ][ E ]
After:  [ A ][ B ][ X ][ C ][ D ][ E ]
          ^ C, D, E all had to MOVE. O(n) time.

Linked List (scattered in memory):
[A|addr2] ---> [B|addr3] ---> [C|None]
They live anywhere. Connected only by pointers.
```

```
Insert X between A and B:
[A|addrX] ---> [X|addrB] ---> [B|addr3] ---> [C|None]
No shifting. Just update 2 pointers. O(1) time.
```

Operation	Array	Linked List	Winner
Insert at start	O(n) — shift all	O(1) — update head	Linked List
Insert at end	O(1) amortized	O(n) without tail ptr	Array
Access by index [i]	O(1) — instant!	O(n) — must walk	Array
Delete from middle	O(n) — shift	O(n) traverse, O(1) delete	Tie
Memory layout	Contiguous (fast cache)	Scattered (slow cache)	Array
Dynamic size	Needs resize + copy	Grows freely	Linked List

SECTION 2 — The Node: Building Block of Everything

What Is a Node?

A node is just a tiny container. Think of it as a box with two compartments: one for the data, and one sticky note that says “the next node lives at THIS address in memory.”

In every type of linked list, the node is the fundamental unit. The only difference between list types is how many pointers each node has.

Node in Python

In Python, we use a class. Each instance of this class IS one node. Indentation is critical in Python — everything inside the class must be indented with 4 spaces (or 1 tab consistently).

```
# Python Node for Singly Linked List
class Node:
    def __init__(self, data): # __init__ is the constructor
        self.info = data     # the value stored in this node
        self.next = None     # pointer to next node (None = no next node yet)

# Creating a node:
node1 = Node(10)           # node1.info = 10, node1.next = None
node2 = Node(20)           # node2.info = 20, node2.next = None
node3 = Node(30)           # node3.info = 30, node3.next = None

# Linking nodes manually:
node1.next = node2         # node1 --> node2
node2.next = node3         # node2 --> node3
# node3.next is still None --> end of list
```

Node in C

In C, we use a struct. A struct is a custom data type that groups multiple values together. We also use a typedef so we can write 'Node' instead of 'struct Node' every time.

```
/* C Node for Singly Linked List */
#include <stdio.h>
#include <stdlib.h> /* needed for malloc() */

typedef struct Node {
    int info;          /* the data stored */
    struct Node* next; /* pointer to the next node */
} Node;

/* Creating a node using malloc (memory allocation): */
Node* create_node(int data) {
    Node* new_node = (Node*)malloc(sizeof(Node)); /* allocate memory */
    new_node->info = data; /* use -> to access member via pointer */
    new_node->next = NULL; /* NULL means no next node */
    return new_node;
}

/* Usage: */
Node* node1 = create_node(10);
Node* node2 = create_node(20);
node1->next = node2; /* node1 --> node2 */
```

Node in C++

C++ can use structs like C, or classes. Here we use a struct with a constructor for cleaner code.

```
// C++ Node for Singly Linked List
#include <iostream>
using namespace std;

struct Node {
    int info;
    Node* next;

    // Constructor (called automatically when creating a Node)
    Node(int data) : info(data), next(nullptr) {}
};

// Creating nodes:
Node* node1 = new Node(10); // new allocates on the heap
Node* node2 = new Node(20);
node1->next = node2; // node1 --> node2
```

Key Rule — Never Lose Head!

You never name nodes node1, node2, node3 etc. in a real program — that would require declaring unlimited variables. Instead, you keep ONE pointer called head that points to the FIRST node. From head, you can reach every other node by following the chain.

SECTION 3 — Singly Linked List

How It Works

The simplest form. Each node only knows about the NEXT node. It is a one-way street. You can only travel forward.

```

head
|
v
[10|-->]  -->  [20|-->]  -->  [30|-->]  -->  NULL
Node 1          Node 2          Node 3 (last)

head points to Node1.
Node1.next points to Node2.
Node2.next points to Node3.
Node3.next = NULL (end of list).
```

TRAVERSAL — Moving Through the List

The most important skill. To visit every node, you need a TEMPORARY pointer (usually called temp or current). You NEVER move head — otherwise you lose the start of your list forever.

```

# Python — Traversal
def traverse(head):
    temp = head                # temp starts at head (first node)
                                # head stays fixed as a bookmark
    while temp != None:        # keep looping until we hit the end
        print(temp.info)       # print current node's data
        temp = temp.next       # move temp to next node

# Example output for [10 --> 20 --> 30 --> None]:
# 10
# 20
# 30
```

```

/* C — Traversal */
void traverse(Node* head) {
    Node* temp = head;         /* temp starts at head */
    while (temp != NULL) {     /* keep going until end */
        printf("%d\n", temp->info); /* print current data */
        temp = temp->next;      /* move to next node */
    }
}
```

INSERT AT START — O(1)

Adding a node at the beginning. Just update two pointers. Very fast because you don't need to traverse anything.

```

# Python — Insert at Start
def insert_at_start(head, data):
    new_node = Node(data)      # Step 1: create the new node
    new_node.next = head       # Step 2: new node points to old head
    head = new_node            # Step 3: head now points to new node
    return head                # return updated head

# Before: head --> [10] --> [20] --> [30] --> None
# After:  head --> [5] --> [10] --> [20] --> [30] --> None
```

```

/* C - Insert at Start */
Node* insert_at_start(Node* head, int data) {
    Node* new_node = create_node(data); /* create new node */
    new_node->next = head;               /* point to old head */
    head = new_node;                   /* update head */
    return head;
}

```

INSERT AT END — $O(n)$

Walk all the way to the last node, then attach the new node there. $O(n)$ because you must traverse the whole list first.

```

# Python - Insert at End
def insert_at_end(head, data):
    new_node = Node(data)
    if head is None:          # list is empty? new node IS the list
        return new_node
    temp = head
    while temp.next != None:  # walk until last node
        temp = temp.next     # (stop condition: temp.next is None)
    temp.next = new_node     # attach new node at the end
    return head

```

```

/* C - Insert at End */
Node* insert_at_end(Node* head, int data) {
    Node* new_node = create_node(data);
    if (head == NULL) return new_node; /* empty list */
    Node* temp = head;
    while (temp->next != NULL) {        /* walk to last node */
        temp = temp->next;
    }
    temp->next = new_node;              /* attach */
    return head;
}

```

INSERT IN MIDDLE (after a specific value)

```

# Python - Insert After Value x
def insert_after(head, data, x):
    new_node = Node(data)
    temp = head
    while temp is not None:            # search for node with value x
        if temp.info == x:             # found it!
            new_node.next = temp.next  # new node points to x's old next
            temp.next = new_node       # x now points to new node
            return head
        temp = temp.next
    print(f"Value {x} not found")
    return head

```

```

/* C - Insert After Value x */
Node* insert_after(Node* head, int data, int x) {
    Node* new_node = create_node(data);
    Node* temp = head;
    while (temp != NULL) {
        if (temp->info == x) {
            new_node->next = temp->next;

```

```

        temp->next = new_node;
        return head;
    }
    temp = temp->next;
}
printf("Value %d not found\n", x);
return head;
}

```

DELETE FROM START — $O(1)$

```

# Python - Delete from Start
def delete_from_start(head):
    if head is None:
        print("List is empty")
        return None
    head = head.next    # head jumps over the first node
    return head         # old first node is now unreachable (garbage
                        # collected)

```

```

/* C - Delete from Start */
Node* delete_from_start(Node* head) {
    if (head == NULL) { printf("Empty list\n"); return NULL; }
    Node* temp = head;    /* save old head to free memory */
    head = head->next;     /* move head forward */
    free(temp);           /* IMPORTANT: free the memory in C! */
    return head;
}

```

DELETE FROM END — $O(n)$

```

# Python - Delete from End
def delete_from_end(head):
    if head is None:          # empty list
        print("List is empty")
        return None
    if head.next is None:     # only one node
        return None
    temp = head
    while temp.next.next != None: # stop at SECOND-TO-LAST node
        temp = temp.next
    temp.next = None          # cut off the last node
    return head

```

```

/* C - Delete from End */
Node* delete_from_end(Node* head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) { free(head); return NULL; }
    Node* temp = head;
    while (temp->next->next != NULL) { /* find second-to-last */
        temp = temp->next;
    }
    free(temp->next);    /* free the last node */
    temp->next = NULL;
    return head;
}

```

DELETE A SPECIFIC VALUE

```
# Python — Delete by Value
def delete_value(head, value):
    if head is None:
        return None
    if head.info == value:          # if head itself is the target
        return head.next          # new head = second node
    temp = head
    while temp.next is not None:   # walk through list
        if temp.next.info == value: # found target ahead
            temp.next = temp.next.next # skip over it
            return head
        temp = temp.next
    print(f"Value {value} not found")
    return head
```

```
/* C — Delete by Value */
Node* delete_value(Node* head, int value) {
    if (head == NULL) return NULL;
    if (head->info == value) {          /* head is the target */
        Node* temp = head;
        head = head->next;
        free(temp);
        return head;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        if (temp->next->info == value) {
            Node* del = temp->next;
            temp->next = temp->next->next;
            free(del);
            return head;
        }
        temp = temp->next;
    }
    printf("%d not found\n", value);
    return head;
}
```

SEARCH — O(n)

```
# Python — Search
def search(head, target):
    temp = head
    position = 1
    while temp is not None:
        if temp.info == target:
            return f"Found {target} at position {position}"
        temp = temp.next
        position += 1
    return f"{target} not found"
```

REVERSE A SINGLY LINKED LIST — Classic Interview Question

You need **THREE** pointers: prev, current, and next_node. The idea is to flip each arrow one by one.

```
# Python — Reverse
def reverse(head):
```

```

prev = None          # will become the new 'tail'
current = head       # starts at head, walks forward

while current is not None:
    next_node = current.next # SAVE next before overwriting
    current.next = prev      # flip the arrow to point backward
    prev = current           # move prev forward
    current = next_node      # move current forward

return prev # prev is now the new head

# Step-by-step visualization:
# [1] --> [2] --> [3] --> None
#
# Iteration 1: prev=None, current=[1]
#   save next=[2], flip [1].next=None, prev=[1], current=[2]
#
# Iteration 2: prev=[1], current=[2]
#   save next=[3], flip [2].next=[1], prev=[2], current=[3]
#
# Iteration 3: prev=[2], current=[3]
#   save next=None, flip [3].next=[2], prev=[3], current=None
#
# Loop ends. prev = [3] = new head
# Result: [3] --> [2] --> [1] --> None

```

```

/* C - Reverse */
Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* current = head;
    Node* next_node = NULL;
    while (current != NULL) {
        next_node = current->next; /* save next */
        current->next = prev;      /* flip arrow */
        prev = current;
        current = next_node;
    }
    return prev; /* new head */
}

```

Complete Singly Linked List Class — Python

Here is the full implementation with all operations combined into one clean class. Notice the consistent 4-space indentation — every method inside the class must be indented one level in.

```

class Node:
    def __init__(self, data):
        self.info = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None # empty list starts with head = None

    # ---- INSERT ----
    def insert_at_end(self, value):
        temp = Node(value)

```



```
        if self.head is None:
            self.head = temp
            return
        t = self.head
        while t.next is not None:    # walk to last node
            t = t.next
        t.next = temp

    def insert_at_start(self, value):
        temp = Node(value)
        temp.next = self.head
        self.head = temp

    def insert_after(self, value, x):
        temp = Node(value)
        t = self.head
        while t is not None:
            if t.info == x:
                temp.next = t.next
                t.next = temp
                return
            t = t.next
        print(f"Value {x} not found")

# ---- DELETE ----
def delete_from_start(self):
    if self.head is None:
        print("List is empty")
        return
    self.head = self.head.next

def delete_from_end(self):
    if self.head is None:
        print("List is empty")
        return
    if self.head.next is None:
        self.head = None
        return
    t = self.head
    while t.next.next is not None:    # stop at second-to-last
        t = t.next
    t.next = None

def delete_value(self, value):
    if self.head is None:
        return
    if self.head.info == value:
        self.head = self.head.next
        return
    t = self.head
    while t.next is not None:
        if t.next.info == value:
            t.next = t.next.next
            return
        t = t.next
    print(f"Value {value} not found")

# ---- UTILITY ----
def search(self, target):
```

```

        t = self.head
        pos = 1
        while t is not None:
            if t.info == target:
                return f"Found {target} at position {pos}"
            t = t.next
            pos += 1
        return f"{target} not found"

    def length(self):
        count = 0
        t = self.head
        while t is not None:
            count += 1
            t = t.next
        return count

    def print_list(self):
        if self.head is None:
            print("List is empty")
            return
        t = self.head
        while t is not None:
            print(f"{t.info} --> ", end="")
            t = t.next
        print("None")

# ---- USAGE ----
ll = SinglyLinkedList()
ll.insert_at_end(10)
ll.insert_at_end(20)
ll.insert_at_end(30)
ll.insert_at_start(5)
ll.print_list()      # 5 --> 10 --> 20 --> 30 --> None
ll.delete_from_end()
ll.print_list()      # 5 --> 10 --> 20 --> None
print(ll.search(10))  # Found 10 at position 2

```

□ SECTION 4 — Doubly Linked List

How It Works

Each node now has TWO pointers: one to the NEXT node and one to the PREVIOUS node. You can travel in both directions. This makes certain operations easier, especially deleting a node when you already have a pointer to it.

```

head
|
v
[None | 10 | -->] <----> [<-- | 20 | -->] <----> [<-- | 30 | None]
    prev data next       prev data next       prev data next

head.prev = None (first node has no previous)
Each middle node has both prev and next.

```

```
Last node's next = None.
```

The Node

```
# Python - Doubly Linked List Node
class Node:
    def __init__(self, data):
        self.info = data
        self.next = None    # points forward
        self.prev = None    # points backward (NEW!)
```

```
/* C - Doubly Linked List Node */
typedef struct Node {
    int info;
    struct Node* next;
    struct Node* prev;    /* NEW: pointer to previous node */
} Node;
```

INSERT AT START

```
# Python - Doubly: Insert at Start
def insert_at_start(head, value):
    temp = Node(value)
    if head is None:
        return temp                # single node, no links needed
    temp.next = head               # new node points forward to old head
    head.prev = temp              # old head points backward to new node
    head = temp                   # head now points to new node
    return head
```

```
/* C - Doubly: Insert at Start */
Node* insert_at_start(Node* head, int value) {
    Node* temp = create_node(value);
    if (head == NULL) return temp;
    temp->next = head;
    head->prev = temp;
    head = temp;
    return head;
}
```

INSERT AT END

```
# Python - Doubly: Insert at End
def insert_at_end(head, value):
    temp = Node(value)
    if head is None:
        return temp
    t = head
    while t.next is not None:    # walk to last node
        t = t.next
    t.next = temp               # last node points to new node
    temp.prev = t               # new node points back to old last
    return head
```

```
/* C - Doubly: Insert at End */
Node* insert_at_end(Node* head, int value) {
    Node* temp = create_node(value);
```

```

    if (head == NULL) return temp;
    Node* t = head;
    while (t->next != NULL) t = t->next;
    t->next = temp;
    temp->prev = t;
    return head;
}

```

DELETE FROM START

```

# Python - Doubly: Delete from Start
def delete_from_start(head):
    if head is None:
        print("List is empty")
        return None
    if head.next is None:      # only one node
        return None
    head = head.next          # move head forward
    head.prev = None          # clear the backward link (no predecessor now)
    return head

```

DELETE FROM END

```

# Python - Doubly: Delete from End
def delete_from_end(head):
    if head is None:
        print("List is empty")
        return None
    if head.next is None:      # only one node
        return None
    t = head
    while t.next is not None:  # walk to last node
        t = t.next
    t.prev.next = None         # second-to-last node's next = None
    return head

# With doubly LL, we can also do this more elegantly:
# last_node = ... (if we have tail pointer)
# last_node.prev.next = None --> O(1)!

```

DELETE A SPECIFIC VALUE

Doubly linked lists make this cleaner because we can update the previous node's pointer directly.

```

# Python - Doubly: Delete by Value
def delete_value(head, value):
    if head is None:
        return None
    t = head
    if t.info == value:        # deleting the head node
        head = t.next
        if head:              # if list is not now empty
            head.prev = None  # new head has no predecessor
        return head
    while t is not None and t.info != value:  # search
        t = t.next
    if t is None:
        print(f"Value {value} not found")

```

```
        return head
    # t is the node to delete
    if t.prev:
        # connect previous to next
        t.prev.next = t.next
    if t.next:
        # connect next back to previous
        t.next.prev = t.prev
    return head
```

Complete Doubly Linked List Class — Python

```
class Node:
    def __init__(self, data):
        self.info = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_start(self, value):
        temp = Node(value)
        if self.head is None:
            self.head = temp
            return
        temp.next = self.head
        self.head.prev = temp
        self.head = temp

    def insert_at_end(self, value):
        temp = Node(value)
        if self.head is None:
            self.head = temp
            return
        t = self.head
        while t.next is not None:
            t = t.next
        t.next = temp
        temp.prev = t

    def delete_from_start(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is None:
            self.head = None
            return
        self.head = self.head.next
        self.head.prev = None

    def delete_from_end(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is None:
            self.head = None
```

```

        return
    t = self.head
    while t.next is not None:
        t = t.next
    t.prev.next = None

def print_forward(self):
    if self.head is None:
        print("List is empty")
        return
    t = self.head
    while t is not None:
        print(f"{t.info} <=> ", end="")
        t = t.next
    print("None")

def print_backward(self):
    if self.head is None:
        print("List is empty")
        return
    t = self.head
    while t.next is not None:    # go to last node first
        t = t.next
    while t is not None:        # now walk backwards using prev
        print(f"{t.info} <=> ", end="")
        t = t.prev
    print("None")

# ---- USAGE ----
dll = DoublyLinkedList()
dll.insert_at_end(10)
dll.insert_at_end(20)
dll.insert_at_end(30)
dll.print_forward()    # 10 <=> 20 <=> 30 <=> None
dll.print_backward()   # 30 <=> 20 <=> 10 <=> None

```

SECTION 5 — Circular Singly Linked List

How It Works

Same as singly, BUT the last node does NOT point to None. Instead, it points BACK to the head. This creates a loop that never ends.

```

head
|
v
[10|-->] --> [20|-->] --> [30|-->] --> [40|-->]
^                                     |
|_____                             |
Last node's next = head (NOT None!)

```

DANGER: A while(temp != None) loop will run FOREVER here!
You need a different stop condition (see below).

Critical Warning

Never use `while temp != None` for circular lists. Since no node ever points to None, this loop runs forever. Instead, use `while temp.next != head` to stop at the last node, or break when you loop back to head.

TRAVERSAL — Circular Safe Loop

```
# Python — Circular Singly: Traversal
def traverse(head):
    if head is None:
        print("List is empty")
        return
    temp = head
    while True:
        print(temp.info)          # do-while style loop
        # process current node
        temp = temp.next
        if temp is head:          # we've gone full circle!
            break

# Alternative (simpler for most operations):
# while temp.next is not head:    # stops AT the last node
#     temp = temp.next
```

```
/* C — Circular Singly: Traversal */
void traverse(Node* head) {
    if (head == NULL) return;
    Node* temp = head;
    do {
        printf("%d\n", temp->info);
        temp = temp->next;
    } while (temp != head);    /* stop when back at start */
}
```

INSERT AT END

Walk to the last node (the one whose next is head), then attach the new node and make it point back to head.

```
# Python — Circular Singly: Insert at End
def insert_at_end(head, value):
    temp = Node(value)
    if head is None:          # first node points to itself
        temp.next = temp
        return temp
    t = head
    while t.next is not head:  # walk to last node
        t = t.next
    t.next = temp             # last node --> new node
    temp.next = head          # new node --> head (keep the circle!)
    return head
```

```
/* C — Circular Singly: Insert at End */
Node* insert_at_end(Node* head, int value) {
    Node* temp = create_node(value);
    if (head == NULL) { temp->next = temp; return temp; }
    Node* t = head;
```

```

while (t->next != head) t = t->next; /* find last node */
t->next = temp;
temp->next = head; /* wrap back to head */
return head;
}

```

INSERT AT START

Tricky: you must **ALSO** update the last node's pointer to point to the new head.

```

# Python - Circular Singly: Insert at Start
def insert_at_start(head, value):
    temp = Node(value)
    if head is None:          # only node, points to itself
        temp.next = temp
        return temp
    t = head
    while t.next is not head: # find last node
        t = t.next
    temp.next = head         # new node --> old head
    head = temp              # update head to new node
    t.next = head            # last node --> new head (important!)
    return head

```

DELETE FROM START

```

# Python - Circular Singly: Delete from Start
def delete_from_start(head):
    if head is None:
        print("List is empty")
        return None
    if head.next is head:    # only one node
        return None
    t = head
    while t.next is not head: # find last node
        t = t.next
    head = head.next         # move head to second node
    t.next = head            # last node now points to new head
    return head

```

Complete Circular Singly Linked List Class — Python

```

class Node:
    def __init__(self, data):
        self.info = data
        self.next = None

class CircularSinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, value):
        temp = Node(value)
        if self.head is None:
            self.head = temp
            temp.next = self.head # points to itself

```



```
        return
    t = self.head
    while t.next is not self.head:
        t = t.next
    t.next = temp
    temp.next = self.head          # wrap back

def insert_at_start(self, value):
    temp = Node(value)
    if self.head is None:
        self.head = temp
        temp.next = self.head
        return
    t = self.head
    while t.next is not self.head:
        t = t.next
    temp.next = self.head
    self.head = temp
    t.next = self.head            # last node --> new head

def delete_from_start(self):
    if self.head is None:
        print("List is empty")
        return
    if self.head.next is self.head: # one node only
        self.head = None
        return
    t = self.head
    while t.next is not self.head:
        t = t.next
    self.head = self.head.next
    t.next = self.head

def delete_from_end(self):
    if self.head is None:
        print("List is empty")
        return
    if self.head.next is self.head:
        self.head = None
        return
    t = self.head
    while t.next.next is not self.head: # stop at 2nd-to-last
        t = t.next
    t.next = self.head              # skip last, point to head

def print_list(self):
    if self.head is None:
        print("List is empty")
        return
    t = self.head
    while True:
        if t.next is not self.head:
            print(f"{t.info} --> ", end="")
        else:
            print(f"{t.info} --> (back to head: {self.head.info})")
        t = t.next
        if t is self.head:
            break
```

```
# ---- USAGE ----
c11 = CircularSinglyLinkedList()
c11.insert_at_end(10)
c11.insert_at_end(20)
c11.insert_at_end(30)
c11.print_list() # 10 --> 20 --> 30 --> (back to head: 10)
```

● SECTION 6 — Circular Doubly Linked List

How It Works

The most feature-rich type. Combines everything: two pointers per node (next + prev) AND the list is circular. The last node's next points to head, AND head's prev points to the last node.

```
head
|
v
.--> [None<-10->] <--> [<-20->] <--> [<-30->] -----
|
'----- head.prev = last node   last.next = head -----'
```

In other words:

```
head.prev = last node (can reach end in O(1)!)
last.next = head      (circular)
```

The Node

```
# Python — Circular Doubly Node (same as doubly)
class Node:
    def __init__(self, data):
        self.info = data
        self.next = None
        self.prev = None
```

INSERT AT END

```
# Python — Circular Doubly: Insert at End
def insert_at_end(head, value):
    temp = Node(value)
    if head is None:
        temp.next = temp      # points to itself (both directions!)
        temp.prev = temp
        return temp
    last = head.prev          # head.prev always = last node (O(1)!)
    last.next = temp          # old last --> new node
    temp.prev = last          # new node <-- old last
    temp.next = head          # new node --> head
    head.prev = temp          # head <-- new node
    return head
```

```
/* C — Circular Doubly: Insert at End */
Node* insert_at_end(Node* head, int value) {
    Node* temp = create_node(value);
```

```

    if (head == NULL) {
        temp->next = temp;
        temp->prev = temp;
        return temp;
    }
    Node* last = head->prev; /* head->prev = last node always */
    last->next = temp;
    temp->prev = last;
    temp->next = head;
    head->prev = temp;
    return head;
}

```

INSERT AT START

```

# Python - Circular Doubly: Insert at Start
def insert_at_start(head, value):
    temp = Node(value)
    if head is None:
        temp.next = temp
        temp.prev = temp
        return temp
    last = head.prev
    temp.next = head          # new node --> old head
    head.prev = temp          # old head <-- new node
    last.next = temp          # last node --> new head
    temp.prev = last          # new head <-- last node
    return temp               # new head!

```

DELETE FROM START

```

# Python - Circular Doubly: Delete from Start
def delete_from_start(head):
    if head is None:
        print("List is empty")
        return None
    if head.next is head:      # only one node
        return None
    last = head.prev
    head = head.next          # move head forward
    head.prev = last          # new head <-- last node
    last.next = head          # last node --> new head
    return head

```

DELETE FROM END — O(1) with circular structure!

```

# Python - Circular Doubly: Delete from End
# Because head.prev = last node, we reach it in O(1)!
def delete_from_end(head):
    if head is None:
        print("List is empty")
        return None
    if head.next is head:      # only one node
        return None
    last = head.prev           # O(1) access to last node!
    second_last = last.prev
    second_last.next = head    # second-last --> head
    head.prev = second_last    # head <-- second-last

```

```
return head
```

Complete Circular Doubly Linked List Class — Python

```
class Node:
    def __init__(self, data):
        self.info = data
        self.next = None
        self.prev = None

class CircularDoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, value):
        temp = Node(value)
        if self.head is None:
            self.head = temp
            temp.next = temp.prev = self.head
            return
        last = self.head.prev
        last.next = temp
        temp.prev = last
        temp.next = self.head
        self.head.prev = temp

    def insert_at_start(self, value):
        temp = Node(value)
        if self.head is None:
            self.head = temp
            temp.next = temp.prev = self.head
            return
        last = self.head.prev
        temp.next = self.head
        self.head.prev = temp
        last.next = temp
        temp.prev = last
        self.head = temp

    def delete_from_start(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is self.head:
            self.head = None
            return
        last = self.head.prev
        self.head = self.head.next
        self.head.prev = last
        last.next = self.head

    def delete_from_end(self):
        if self.head is None:
            print("List is empty")
            return
        if self.head.next is self.head:
```

```

        self.head = None
        return
    last = self.head.prev          # O(1) access!
    second_last = last.prev
    second_last.next = self.head
    self.head.prev = second_last

def delete_value(self, value):
    if self.head is None:
        return
    if self.head.info == value:
        self.delete_from_start()
        return
    t = self.head.next
    while t is not self.head:
        if t.info == value:
            t.prev.next = t.next
            t.next.prev = t.prev
            return
        t = t.next
    print(f"Value {value} not found")

def print_list(self):
    if self.head is None:
        print("List is empty")
        return
    t = self.head
    while True:
        if t.next is not self.head:
            print(f"{t.info} <=> ", end="")
        else:
            print(f"{t.info} <=> (back to head: {self.head.info})")
        t = t.next
        if t is self.head:
            break

# ---- USAGE ----
cdll = CircularDoublyLinkedList()
cdll.insert_at_end(10)
cdll.insert_at_end(20)
cdll.insert_at_end(30)
cdll.print_list()    # 10 <=> 20 <=> 30 <=> (back to head: 10)
cdll.delete_from_end()
cdll.print_list()    # 10 <=> 20 <=> (back to head: 10)

```

□ SECTION 7 — Time & Space Complexity Summary

Singly Linked List

Operation	Time	Explanation
Access by index	$O(n)$	Must walk from head each time

Search for value	$O(n)$	Must check every node in worst case
Insert at head	$O(1)$	Just update 1 pointer
Insert at tail	$O(n)$	Must walk to end first
Insert at tail (tail ptr)	$O(1)$	Direct access if tail maintained
Delete at head	$O(1)$	Just move head forward
Delete at tail	$O(n)$	Must find second-to-last node
Delete in middle	$O(n)$	Must find the node first
Space per node	$O(1)$	Data + 1 pointer

Doubly vs Singly — Key Differences

Operation	Singly	Doubly	Why Different?
Delete known node (have pointer)	$O(n)$	$O(1)$	Doubly has prev pointer
Delete from tail (tail ptr)	$O(n)$	$O(1)$	Doubly: tail.prev gives 2nd-to-last
Traverse backward	Not possible	$O(n)$	Doubly has prev pointers
Space per node	Data + 1 ptr	Data + 2 ptrs	Extra prev pointer

Circular Advantage

Operation	Regular	Circular	Why?
Insert at start	$O(1)$	$O(n) / O(1)^*$	* $O(1)$ if tail pointer maintained
Access last node	$O(n)$	$O(1)$ (doubly circular)	head.prev = last node
Wrap-around traversal	Requires restart	Natural	Just keep following next

SECTION 8 — All 4 Types: Quick Comparison

Feature	Singly	Doubly	Circular Singly	Circular Doubly
Direction	Forward only	Both ways	Forward + loops	Both + loops
prev pointer	No	Yes	No	Yes
Last node points to	None	None	head	head
head.prev	Not applicable	None	Not applicable	Last node

Can traverse backward	No	Yes	No	Yes
Infinite loop risk	No	No	Yes (careful!)	Yes (careful!)
Memory per node	Low (1 ptr)	Medium (2 ptrs)	Low (1 ptr)	Medium (2 ptrs)
Best for	Simple queues, hash chaining	LRU cache, browser history	Round-robin scheduling	Advanced caches, window managers

SECTION 9 — Real-World Use Cases

Singly Linked List — Where Is It Used?

- **Hash Table Chaining:** When two keys hash to the same bucket, a singly linked list stores the chain. This is how Python's dict and Java's HashMap handle collisions.
- **Memory Allocators:** The OS keeps a singly linked list of free memory blocks. malloc() walks this list to find space. Used in Linux kernel, glibc, jemalloc (Firefox, Meta).
- **Symbol Tables in Compilers:** GCC and Clang store variable names in hash tables with linked list chaining in each bucket.
- **Blockchain:** Each block contains a hash of the previous block. One-way chain, just like singly linked list.

Doubly Linked List — Where Is It Used?

- **Browser Back/Forward:** Chrome, Firefox, Safari all implement browser history as a doubly linked list. Back = prev, Forward = next.
- **LRU Cache:** The most famous use case. Doubly linked list + hash map = O(1) get and put. Used by Redis, Memcached, CDN caches at Cloudflare.
- **Text Editor Cursor:** Moving cursor left/right in Vim, Emacs. Each character or line is a node.
- **Linux Kernel Thread Scheduler:** The kernel's run queue is a doubly linked list. Threads added and removed from both ends efficiently.
- **Undo/Redo Systems:** VS Code, Photoshop, Google Docs, Microsoft Word all use doubly linked lists for Ctrl+Z / Ctrl+Y.

Circular Singly — Where Is It Used?

- **Round Robin CPU Scheduling:** OS cycles through all processes infinitely. After the last, it loops back to first. Linux scheduler, Windows task scheduler, FreeRTOS.
- **Multiplayer Games:** Player 1 -> Player 2 -> Player 3 -> back to Player 1. Chess.com, online card games.
- **Music Player Repeat All:** VLC, Spotify, YouTube Music — when repeat is on, after the last song it goes back to first.

- **Circular Buffers in Networking:** Network drivers use circular buffers for incoming packets. When full, overwrites oldest data. Used in Linux network drivers, embedded/IoT systems.

Circular Doubly — Where Is It Used?

- **Advanced LRU Cache:** Circular structure avoids None boundary checks. Used in database buffer pools (PostgreSQL, MySQL InnoDB).
- **Fibonacci Heaps:** Used inside Dijkstra's algorithm for GPS navigation and shortest path (Google Maps, OSPF routing protocol).
- **Window Managers:** Alt+Tab cycles through windows in a circle, forward and backward. Used by Linux GNOME/KDE, Windows DWM.
- **Real-Time OS (RTOS):** VxWorks (Boeing, NASA), QNX (cars), FreeRTOS use circular doubly lists for task scheduling that needs both priority upgrades and forward scheduling.

SECTION 10 — Common Mistakes & Gotchas

Mistake 1: Moving head Instead of temp

The Rule

head is your permanent bookmark to the first node. NEVER move it during traversal. Create a temp pointer and move that instead.

```
# Python
# WRONG — you lose your list forever!
while head != None:
    print(head.info)
    head = head.next    # head is permanently gone after this

# CORRECT — use a temp pointer
temp = head             # temp starts at head
while temp != None:
    print(temp.info)
    temp = temp.next    # only temp moves, head stays fixed
```

Mistake 2: Forgetting None Check Before Accessing .info

```
# Python
# WRONG — crashes if list is empty (head is None)
print(head.info)    # AttributeError: 'NoneType' has no attribute 'info'

# CORRECT — always guard first
if head is not None:
    print(head.info)

# Also: guard before accessing .next.info (two levels deep)
if head is not None and head.next is not None:
    print(head.next.info)
```


Mistake 3: Infinite Loop in Circular Lists

```
# Python
# WRONG — temp never becomes None in a circular list, loops forever!
while temp != None:
    temp = temp.next

# CORRECT option 1 — break when you return to head
temp = head
while True:
    # do something
    temp = temp.next
    if temp is head:    # 'is' checks same object, not just equal value
        break

# CORRECT option 2 — stop at last node (for insert/delete)
while temp.next is not head:
    temp = temp.next    # temp ends up AT the last node
```

Mistake 4: Losing the Next Node During Deletion

```
# Python
# WRONG — you destroy the reference before using it
temp.next = None    # oops! temp.next.next is now unreachable
temp.next = temp.next.next    # this line can no longer work

# CORRECT — save first, then update
saved = temp.next.next    # save what comes AFTER the node to delete
temp.next = saved    # now safely update the pointer
```

Mistake 5: Wrong Stop Condition for Traversal

```
# Python
# These two loops are DIFFERENT. Know which one you need!

# Option 1: visits ALL nodes including the last one
while temp != None:    # stops AFTER last node (temp becomes None)
    process(temp)
    temp = temp.next

# Option 2: stops AT the last node
while temp.next != None:    # stops WHEN temp IS the last node
    temp = temp.next
# After this loop, temp is pointing to the last node
# Use this when you need to insert or link AT the end!
```

Mistake 6: Forgetting to free() memory in C

C-Specific Warning

Python has garbage collection — when you stop pointing to a node, it gets cleaned up automatically. C does NOT have this. Every node you create with `malloc()` MUST be freed with `free()` when deleted. Forgetting this causes memory leaks.

```
/* C — WRONG: memory leak! */
```

```

head = head->next;    /* old head is now unreachable but memory never freed */

/* C - CORRECT */
Node* temp = head;    /* save pointer to old head */
head = head->next;     /* move head */
free(temp);           /* now free the memory of old head */

```

Mistake 7: Using == Instead of 'is' for Object Comparison in Python

```

# Python
# WRONG for circular list check: == compares VALUES
if temp == head:      # True if temp.info == head.info (not same object!)
    break

# CORRECT: 'is' checks if they are the SAME object in memory
if temp is head:      # True only if temp and head point to SAME node
    break

# Example of why this matters:
# If two different nodes both hold value 10, == gives True for both
# but is only gives True for the actual head node.

```

□ FINAL SUMMARY

The Golden Rule of Linked Lists

head is sacred. Never move it. Use a temp pointer to traverse. For circular lists: never use `while temp != None`. Use `while temp.next != head` or break when `temp is head`. In C: always free() deleted nodes to prevent memory leaks.

Linked lists are the foundation of dynamic data structures. Every hash map, every browser history, every LRU cache, every OS scheduler — somewhere underneath, a linked list is doing the heavy lifting. Master them, and you master the backbone of systems programming.

عَلِيمٌ عِلْمِ ذِي كُلِّ وَفَوْقَ

"Above every person of knowledge is one more knowing." — Surah Yusuf, 12:76

Keep learning. The chain never ends. □